

**Name: A. Raja**

**Reg No: 192365033**

**Course Code: CSA1718**

## **PROGRAM – 1**

### **8-Puzzle Problem**

#### **Aim**

To solve the **8-Puzzle Problem** using the Best First Search approach.

#### **Algorithm**

1. Define the initial state and goal state.
2. Find the position of the blank tile (0).
3. Generate all possible moves of the blank tile.
4. Select the next state based on the search strategy.
5. Continue until the goal state is reached.
6. Display the solution path.

#### **Python Program**

```
from queue import PriorityQueue
```

```
goal = [[1,2,3],  
        [4,5,6],  
        [7,8,0]]
```

```
initial = [[1,2,3],  
           [4,0,6],  
           [7,5,8]]
```

```
def heuristic(state):  
    count = 0  
    for i in range(3):
```

```

        for j in range(3):
            if state[i][j] != 0 and state[i][j] != goal[i][j]:
                count += 1
    return count

pq = PriorityQueue()
pq.put((heuristic(initial), initial))

visited = []

while not pq.empty():
    h, state = pq.get()

    if state == goal:
        print("Goal State Reached")
        for row in state:
            print(row)
        break

    if state not in visited:
        visited.append(state)

    x = y = 0
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                x, y = i, j

    moves = [(-1,0),(1,0),(0,-1),(0,1)]

    for dx, dy in moves:
        nx = x + dx
        ny = y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:
            new = [row[:] for row in state]
            new[x][y], new[nx][ny] = new[nx][ny], new[x][y]

            if new not in visited:
                pq.put((heuristic(new), new))

```

## Expected Output

Goal State Reached

[1, 2, 3]

[4, 5, 6]

[7, 8, 0]

# PROGRAM – 2

## 8-Queen Problem

### Aim

To place **8 queens** on an **8 × 8 chessboard** such that no two queens attack each other.

### Algorithm

1. Start from the first column.
2. Place a queen in a safe row.
3. Check rows and diagonals for conflicts.
4. If safe, place the queen and move to the next column.
5. If no safe position exists, backtrack.
6. Repeat until all eight queens are placed.

### Python Program

```
N = 8
```

```
board = [[0 for i in range(N)] for j in range(N)]
```

```
def isSafe(board, row, col):
```

```
    for i in range(col):
        if board[row][i] == 1:
            return False
```

```
    i = row
    j = col
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1
```

```
    i = row
    j = col
```

```

while i < N and j >= 0:
    if board[i][j] == 1:
        return False
    i += 1
    j -= 1

return True

def solve(board, col):

    if col >= N:
        return True

    for i in range(N):

        if isSafe(board, i, col):

            board[i][col] = 1

            if solve(board, col + 1):
                return True

            board[i][col] = 0

    return False

solve(board, 0)
print("Solution")

for row in board:
    print(row)

```

## Expected Output

```

[1, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 1, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 1]
[0, 0, 0, 0, 0, 1, 0, 0]
[0, 0, 1, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 1, 0]
[0, 1, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 1, 0, 0, 0, 0]

```